
Uso de redes neuronales para comprimir y visualizar datos multidimensionales

Introducción

Proporcionar una mejor comprensión de los datos multidimensionales es una tarea importante. Este pequeño documento muestra cómo completar esta tarea utilizando redes neuronales de retropropagación (perceptrón multicapa).

Una de las partes de EDA (Exploratory Data Analysis) es la visualización de datos multidimensionales, que requiere un algoritmo eficaz para mapear nD datos en un plano 2D.

Hay muchas técnicas, incluidos los mapas autoorganizados de Kohonen, SVD, PCA, etc., pero a veces son difíciles de interpretar. Aquí se presenta otro, que parece más sencillo de entender. Esta técnica ilustrará el conocido conjunto de datos de Iris.

Análisis

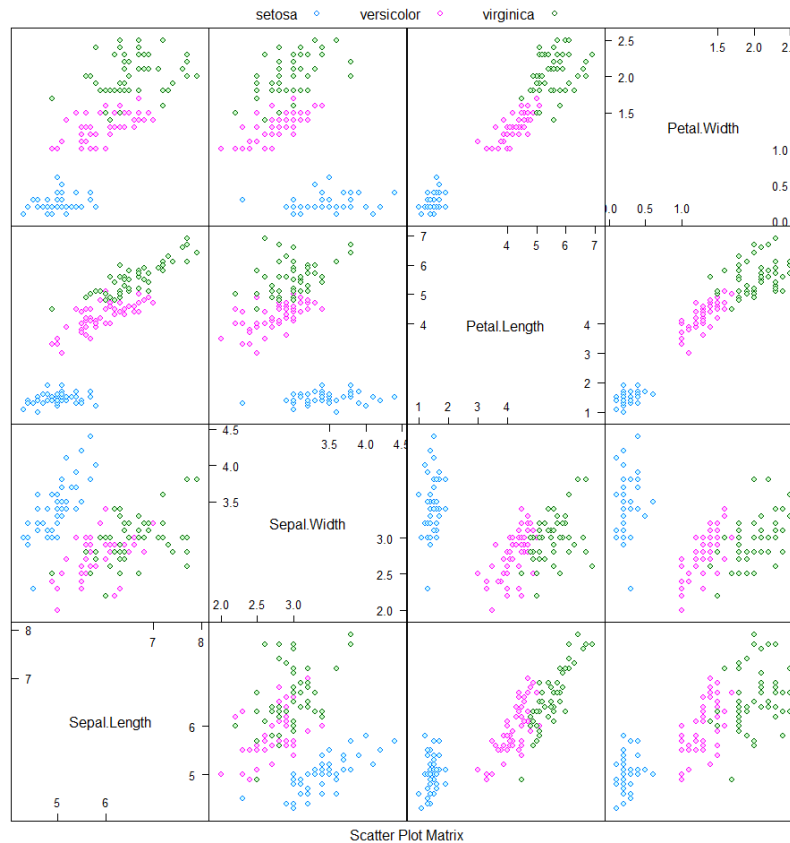
El conjunto de datos Iris contiene cuatro características de entrada (longitud del sépalo, ancho del sépalo, longitud del pétalo, ancho del pétalo) y una salida: especie.

Por lo tanto, es el espacio de entrada 4D el que no se puede visualizar directamente. Usando gráficos 2D, podemos mostrar solo relaciones de pares como se indica a continuación.

```
> library(neuralnet)
> library(caret)
> library(ggplot2)

> # Imprime la grafica de pares simples
> featurePlot(x = iris[, 1:4], y = iris$Species, plot = "pairs", pch = 21, auto.key = list(columns = 3))
```

Parece bastante comprensible, pero, por ejemplo, no podemos asegurarnos de que todo el espacio de entrada contenga clústeres. Incluso los datos 4D pueden confundir. Además, es aún más difícil sacar conclusiones si tenemos más dimensiones



Las redes neuronales de retropropagación pueden reducir eficazmente la dimensionalidad de los datos. Para hacer esto, necesitamos pasar los datos a través del “cuello de botella” con una dimensión más baja que los datos de origen. Esta red neuronal debe tener cinco capas: capa de entrada, primera capa oculta para realizar la compresión de datos, segunda capa oculta “cuello de botella”, tercera capa oculta para realizar la descompresión de datos y una capa de salida con el mismo tamaño que una capa de entrada. Así, este tipo de redes neuronales tiene una arquitectura simétrica.

En nuestro caso, el “cuello de botella” contendrá solo dos neuronas, que son necesarias para visualizar datos 2D. Y la red permitirá que solo los datos más importantes pasen a través de esta capa. Más adelante veremos que estos datos solo son necesarios para restaurar la distribución de los datos de origen.

Primero, creamos un conjunto de datos de entrenamiento con las mismas entradas y salidas y hacemos el preproceso simple (normalización):

```
> train <- iris
> train <- train[,1:4]
> train <- cbind(train, train)
> colnames(train) <- c("SL.I", "SW.I", "PL.I", "PW.I", # Inputs from iris dataset...
```

```

+           "SL.O", "SW.O", "PL.O", "PW.O") # and the same outputs
> # Procesar previamente el conjunto de datos de entrenamiento con el paquete caret
> preProcValues <- preProcess(train, method = c("range"))
> train2 <- predict(preProcValues, train)

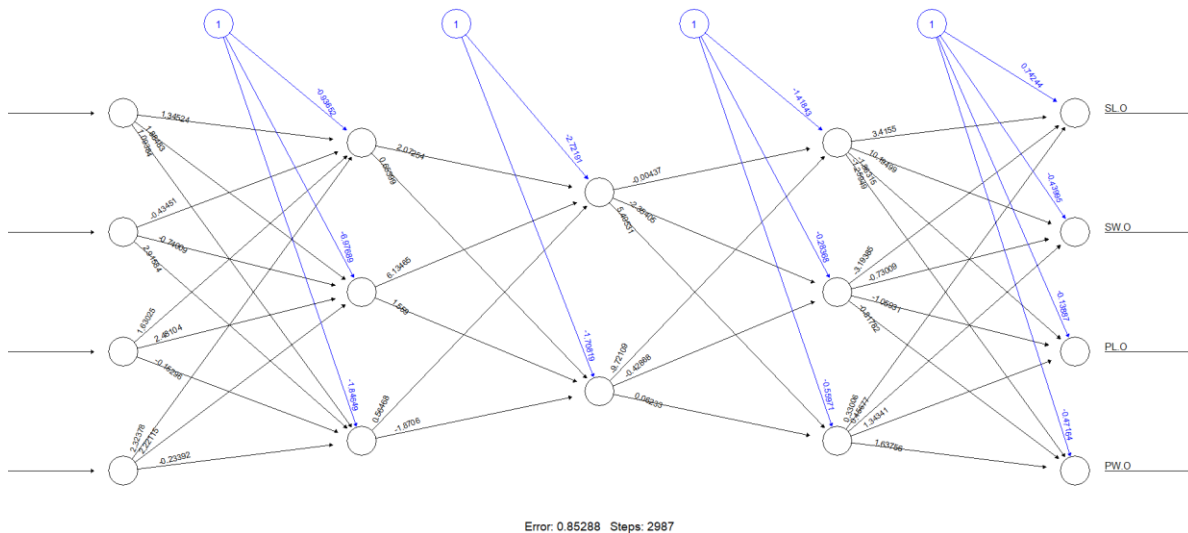
```

A continuación, creamos el modelo de red neuronal que comprime los datos de origen 4D en un conjunto de datos 2D:

```

> set.seed(1)
> model2d <- neuralnet(SL.O + SW.O + PL.O + PW.O ~ SL.I + SW.I + PL.I + PW.I,
+           train2, hidden=c(3,2,3), algorithm = 'rprop+', threshold = 0.01)
> # print (paste("Mean square error = ", model2d$result.matrix[1]))
> plot(model2d, rep = 1)

```

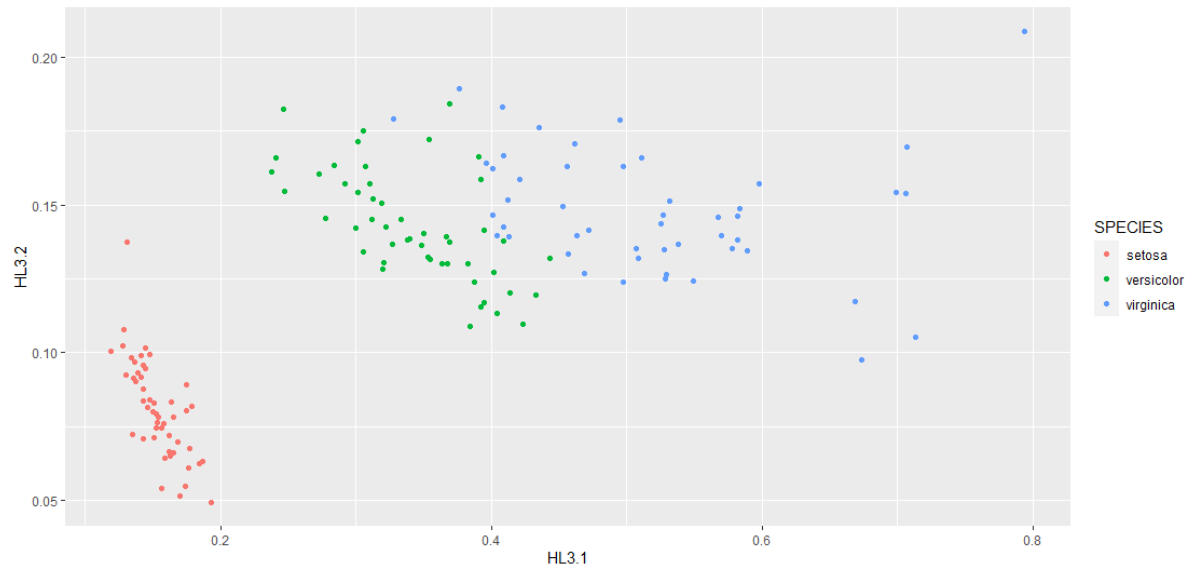


En tercer lugar, obtenemos los valores de activación de la capa oculta de en medio para crear un nuevo conjunto de datos 2D comprimido:

```

> result2d <- compute(model2d, train2[,1:4]) # Run input set through the neural network
> out2d <- as.data.frame(result2d$neurons[[3]][,2:3]) # Get the 2D-data from middle layer
> out2d <- cbind(out2d, iris[,5]) # Append labels from source dataset
> colnames(out2d) <- c("HL3.1", "HL3.2", "SPECIES")
> ggplot(out2d, aes(x=HL3.1, y=HL3.2, color=SPECIES)) + geom_point() # Visualize it

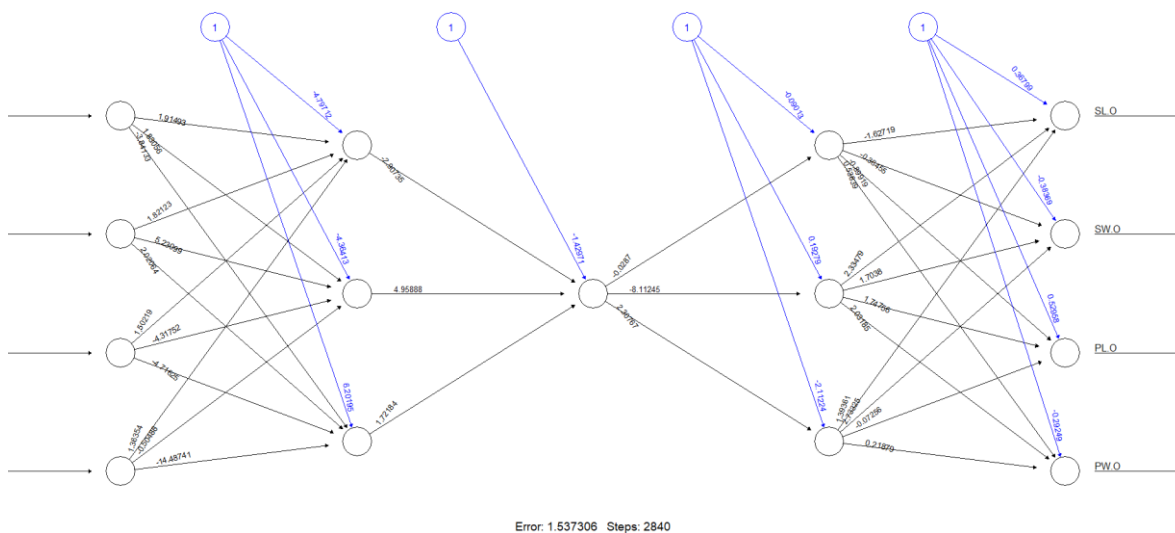
```



Por lo tanto, hemos comprimido el espacio de entrada 4D en un conjunto de datos 2D sin significado físico, lo hemos visualizado y etiquetado. Como podemos ver, los tres grupos que se corresponden con las especies de iris se pueden determinar visualmente con claridad.

¡Intentemos enviar los datos a través de una sola neurona!

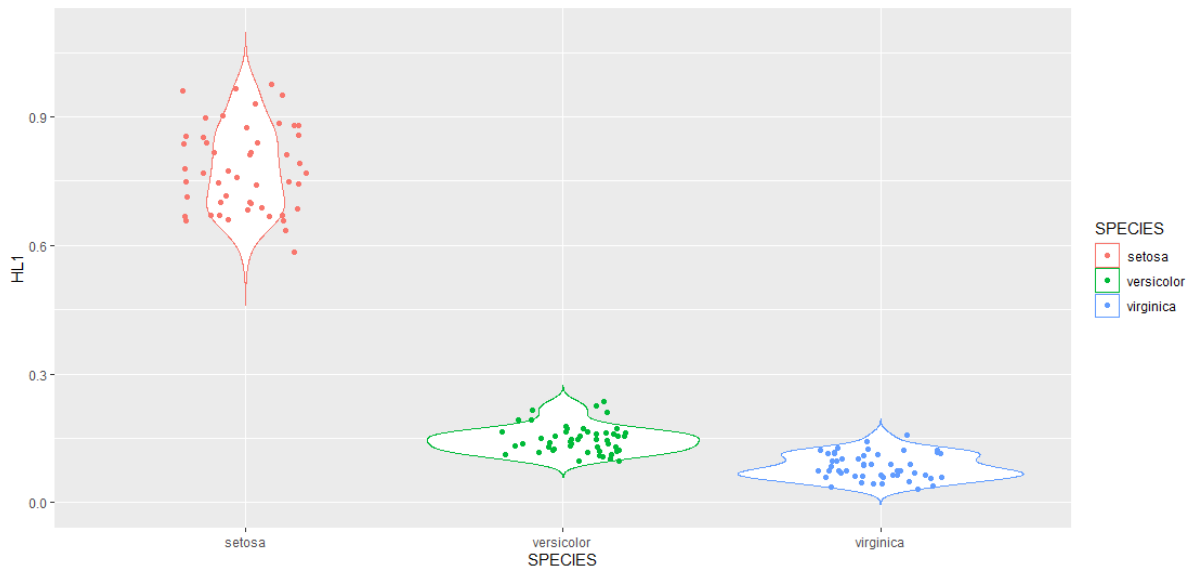
```
> set.seed(1)
> model1d <- neuralnet(SL.O + SW.O + PL.O + PW.O ~ SL.I + SW.I + PL.I + PW.I,
+                       train2, hidden=c(3,1,3), algorithm = 'rprop+', threshold = 0.01)
> # print (paste("Mean square error = ", model1d$result.matrix[1]))
> plot(model1d, rep=1)
```



```

> result1d <- compute(model1d, train2[,1:4]) #Run them through the neural network
> out1d <- as.data.frame(result1d$neurons[[3]][,2])
> out1d <- cbind(out1d, iris[,5])
> colnames(out1d) <- c("HL1", "SPECIES")
> ggplot(out1d, aes(x=SPECIES, y=HL1, color=SPECIES)) +
+   geom_violin(trim = FALSE) +
+   geom_jitter(position=position_jitter(0.2))

```



Como podemos ver, funciona peor. Este modelo tiene un error cuadrático medio mayor. Y su agrupación no se ve bien; allí no podemos ver tres agrupaciones claras: los datos según las especies versicolor y virginica se cruzan en un intervalo de valores de 1-1,5 de la neurona del “cuello de botella”.

Conclusión

El ejemplo anterior muestra que las redes neuronales de retropropagación pueden ser muy útiles para comprimir y visualizar datos multidimensionales. Y esta técnica puede ser más clara que algunos métodos conocidos. El uso del modelo 2D aplicado al conjunto de datos de iris nos dio una buena comprensión de los grupos de datos existentes. En comparación, el modelo 1D se veía peor, pero **en algunos casos incluso el modelo 1D puede tener sentido**. Por ejemplo, en la tarea de modelar los precios de la vivienda, los datos de solo la neurona media (el “cuello de botella”) **podrían generalizar un conjunto de los factores más importantes que afectan el precio**.